

Assignment 1: Write a program to demonstrate **Row Transposition Cipher**

Code:

```
# Assignment 1: Row Transposition Cipher

# Read plaintext from file and write ciphertext to another file

def row_transposition_encrypt(plaintext, key):
    key_digits = list(key)
    key_order = sorted(range(len(key_digits)), key=lambda x: key_digits[x])
    # Remove spaces/newlines for matrix placement
    plaintext = plaintext.replace(" ", "").replace("\n", "")
    # Calculate rows needed
    cols = len(key)
    rows = (len(plaintext) + cols - 1) // cols
    # Fill matrix row-wise
    matrix = []
    index = 0
    for r in range(rows):
        row = []
        for c in range(cols):
            if index < len(plaintext):
                row.append(plaintext[index])
            else:
                row.append('X') # padding
            index += 1
```

```

        matrix.append(row)
# Read matrix column-wise based on key order
ciphertext = ""
for col in key_order:
    for row in matrix:
        ciphertext += row[col]
    return ciphertext
# File IO
input_file = "input.txt"
output_file = "row_trans_output.txt"
key = input("Enter Row Transposition key (e.g. 431256): ")
with open(input_file, "r") as f:
    plaintext = f.read()
ciphertext = row_transposition_encrypt(plaintext, key)
with open(output_file, "w") as f:
    f.write(ciphertext)
print("Row Transposition Cipher Encryption Completed!")

```

Output:

Enter Row Transposition key (e.g. 431256): 241356

Row Transposition Cipher Encryption Completed!

Plain text: my name is meet shah

Cipher text: nmamisaehyshmeXetX

Assignment 2: Write a program to demonstrate Polybius Cipher

Code:

```
# Assignment 2: Polybius Cipher (File input/output)
```

```
polybius_square = {  
    'A': "11", 'B': "12", 'C': "13", 'D': "14", 'E': "15",  
    'F': "21", 'G': "22", 'H': "23", 'I': "24", 'J': "24", 'K': "25",  
    'L': "31", 'M': "32", 'N': "33", 'O': "34", 'P': "35",  
    'Q': "41", 'R': "42", 'S': "43", 'T': "44", 'U': "45",  
    'V': "51", 'W': "52", 'X': "53", 'Y': "54", 'Z': "55"  
}
```

```
def polybius_encrypt(plaintext):  
    plaintext = plaintext.upper().replace(" ", "")  
    ciphertext = ""  
  
    for ch in plaintext:  
        if ch in polybius_square:  
            ciphertext += polybius_square[ch] + " "  
        else:  
            ciphertext += ch  
  
    return ciphertext.strip()  
  
input_file = "input.txt"
```

```
output_file = "poly_output.txt"

with open(input_file, "r") as f:
    plaintext = f.read()

ciphertext = polybius_encrypt(plaintext)

with open(output_file, "w") as f:
    f.write(ciphertext)

print("Polybius Cipher Encryption Completed!")
```

Output:

Polybius Cipher Encryption Completed!

Plain text: my name is meet shah

Cipher text : 32 54 33 11 32 15 24 43 32 15 15 44 43 23 11 23

Assignment 3: Write a program to demonstrate the Rotor Cipher

Code:

```
# Assignment 3: Rotor Cipher Demonstration
# Simple Rotor Implementation (Classroom Model)
```

```
def rotate_char(ch, shift):
    if ch.isalpha():
        base = 'A' if ch.isupper() else 'a'
```

```
    return chr((ord(ch) - ord(base) + shift) % 26 + ord(base))  
    return ch
```

```
def rotor_cipher_encrypt(plaintext, num_rotors, angles):  
    ciphertext = plaintext  
  
    for i in range(num_rotors):  
        shift = angles[i] % 26  
        new_text = ""  
        for ch in ciphertext:  
            new_text += rotate_char(ch, shift)  
        ciphertext = new_text # output of one rotor becomes input to next  
  
    return ciphertext
```

```
# I/O  
input_file = "input.txt"  
output_file = "rotor_output.txt"
```

```
with open(input_file, "r") as f:  
    plaintext = f.read()
```

```
num_rotors = int(input("Enter number of rotors: "))
```

```
angles = []
```

```
for i in range(num_rotors):  
    ang = int(input(f"Enter angle for rotor {i+1}: "))  
    angles.append(ang)  
  
ciphertext = rotor_cipher_encrypt(plaintext, num_rotors, angles)  
  
with open(output_file, "w") as f:  
    f.write(ciphertext)  
  
print("Rotor Cipher Encryption Completed!")
```

Output:

Enter number of rotors: 3

Enter angle for rotor 1: 30

Enter angle for rotor 2: 45

Enter angle for rotor 3: 60

Rotor Cipher Encryption Completed!

Plain Text: my name is meet shah

Cipher Text: rd sfrj nx rjyy xmfm